

Ejercicios: funciones que procesan árboles
Unidad 0: Herramientas y preliminares

Eduardo Acuña Yeomans

Lenguajes de Programación
Semestre 2026-2

Lic. en Ciencias de la Computación
Universidad de Sonora

Sesión 5

Esta hoja no se entrega y no se califica. Al final hay una tabla de pruebas con su resultado esperado: cuando termines, tecléalas todas.

Trabaja sobre el archivo `u0-calc.rkt` que está en Teams. Ya trae la gramática, los cuatro structs, `calc` y `sub-exp`.

```
Exp ::= Num
      | add(Exp, Exp)
      | mul(Exp, Exp)
      | neg(Exp)
```

Para no escribir árboles enormes a cada rato, define primero estos cuatro:

```
(define p (add-exp (num-exp 3) (mul-exp (num-exp 2) (num-exp 4))))
(define q (neg-exp (add-exp (num-exp 1) (num-exp 2))))
(define r (max-exp (add-exp (num-exp 2) (num-exp 3)) ; después del ejercicio 1
                  (mul-exp (num-exp 2) (num-exp 2))))
(define s (mul-exp (add-exp (num-exp 1) (num-exp 2))
                  (add-exp (num-exp 3) (num-exp 4))))
```

Extender el lenguaje

1. **max** Agrega una operación que devuelva el mayor de dos valores. Son tres pasos, los mismos de siempre: escribe la producción nueva, define el struct, agrega el caso en `calc`. Llámala `max-exp`, con campos `izq` y `der`.

```
(calc (max-exp (num-exp 3) (num-exp 5))) = 5
(calc r)                                = 5
```

Sugerencia: qué operación de Racket te sirve

`(max 3 5)` ya existe y devuelve 5. Lo que tienes que decidir es qué le pasas: ¿los campos, o algo que sacas de ellos?

2. Sin escribir código La resta la agregamos en clase de dos formas. ¿Se podría agregar `max` de la segunda forma, es decir traduciéndola a lo que ya existe, sin tocar `calc`? Contéstalo en papel y explica por qué.

Otras funciones sobre el mismo árbol

Las tres que siguen no evalúan nada: recorren el árbol y devuelven otra cosa. Todas se escriben igual: un `match`, un caso por variante, una llamada recursiva por cada campo que sea una expresión.

3. cuantos-nodos Cuenta cuántos nodos tiene el árbol, contando todos: los números y las operaciones.

```
(cuantos-nodos (num-exp 7)) = 1
(cuantos-nodos p)           = 5
(cuantos-nodos q)           = 4
```

Sugerencia: por qué p tiene 5

`p` es `add(3, mul(2, 4))`. Los nodos son: el `add`, el `3`, el `mul`, el `2` y el `4`. Dibújalo si no lo ves.

4. profundidad Cuántos niveles tiene el árbol. Un número suelto tiene profundidad 1.

```
(profundidad (num-exp 7)) = 1
(profundidad p)           = 3
(profundidad s)           = 3
```

Sugerencia: la diferencia con la anterior

En `cuantos-nodos` sumabas lo de las dos ramas. Aquí no se suman: solo cuenta la más honda. (`max a b`) otra vez.

5. a-texto Devuelve la cadena que le corresponde al árbol, lo contrario de lo que hace un parser.

```
(a-texto p) = "add(3, mul(2, 4))"
(a-texto q) = "neg(add(1, 2))"
```

Sugerencia: dos funciones de Racket que vas a necesitar

`(number->string 3)` da `"3"`, y `(string-append "a" "b" "c")` pega cadenas. Fíjate en la coma y el espacio de la salida esperada: tienen que estar exactamente así.

6. Sin escribir código ¿Qué relación hay entre `a-texto` y el `parse` que veremos más adelante? Si le aplicas uno y luego el otro a la misma expresión, ¿qué deberías obtener?

Pruebas

Teclea estos catorce casos. Todos tienen que dar exactamente esto.

<code>(calc p)</code>	<code>= 11</code>
<code>(calc q)</code>	<code>= -3</code>
<code>(calc r)</code>	<code>= 5</code>
<code>(calc s)</code>	<code>= 21</code>
<code>(calc (max-exp (num-exp 3) (num-exp 5)))</code>	<code>= 5</code>
<code>(cuantos-nodos (num-exp 7))</code>	<code>= 1</code>
<code>(cuantos-nodos p)</code>	<code>= 5</code>
<code>(cuantos-nodos q)</code>	<code>= 4</code>
<code>(cuantos-nodos r)</code>	<code>= 7</code>
<code>(profundidad (num-exp 7))</code>	<code>= 1</code>
<code>(profundidad p)</code>	<code>= 3</code>
<code>(profundidad s)</code>	<code>= 3</code>
<code>(a-texto p)</code>	<code>= "add(3, mul(2, 4))"</code>
<code>(a-texto r)</code>	<code>= "max(add(2, 3), mul(2, 2))"</code>

Soluciones

Clave de Respuesta: extender el lenguaje

La producción nueva es `max(Exp, Exp)`, y de ahí salen las otras dos piezas:

```
(struct max-exp (izq der) #:transparent)

;; el caso que se agrega a calc
[(max-exp izq der) (max (calc izq) (calc der))]
```

Igual que con `add` y `mul`: a `max` hay que pasarle los *valores* de los campos, no los campos.

Ejercicio 2. No se puede. `sub` salió sin tocar `calc` porque restar se podía decir con lo que ya había (sumar y tomar el negativo). Con `max` no hay forma de decir “el mayor de los dos” combinando sumas, productos y negativos: hace falta comparar, y comparar no está en el lenguaje. Cuando algo no se puede traducir a lo que ya existe, el único camino es agregarlo de verdad.

Clave de Respuesta: otras funciones sobre el mismo árbol

```
(define (cuantos-nodos e)
  (match e
    [(num-exp _)      1]
    [(add-exp izq der) (+ 1 (cuantos-nodos izq) (cuantos-nodos der))]
    [(mul-exp izq der) (+ 1 (cuantos-nodos izq) (cuantos-nodos der))]
    [(neg-exp cuerpo)  (+ 1 (cuantos-nodos cuerpo))]
    [(max-exp izq der) (+ 1 (cuantos-nodos izq) (cuantos-nodos der))]))

(define (profundidad e)
  (match e
    [(num-exp _)      1]
    [(add-exp izq der) (+ 1 (max (profundidad izq) (profundidad der)))]
    [(mul-exp izq der) (+ 1 (max (profundidad izq) (profundidad der)))]
    [(neg-exp cuerpo)  (+ 1 (profundidad cuerpo))]
    [(max-exp izq der) (+ 1 (max (profundidad izq) (profundidad der)))]))

(define (a-texto e)
  (match e
    [(num-exp n) (number->string n)]
    [(add-exp izq der)
     (string-append "add(" (a-texto izq) ", " (a-texto der) ")")]
    [(mul-exp izq der)
     (string-append "mul(" (a-texto izq) ", " (a-texto der) ")")]
    [(neg-exp cuerpo)
     (string-append "neg(" (a-texto cuerpo) ")")]
    [(max-exp izq der)
     (string-append "max(" (a-texto izq) ", " (a-texto der) ")")]))
```

En `cuantos-nodos` y `profundidad` el caso del número usa el comodín: el valor no importa, solo que sea un número. Y fíjate en que las tres funciones tienen exactamente la misma forma que `calc`: cinco casos, uno por variante, con una llamada recursiva por cada campo que es una expresión. Cambia lo que hacen, no cambia cómo están armadas.

Clave de Respuesta: la pregunta del ejercicio 6

a-texto va del árbol al texto. parse irá del texto al árbol. Son inversas: si tomas un árbol, lo conviertes a texto y lo vuelves a analizar, deberías recuperar el mismo árbol.

Más adelante en el curso, cuando tengas el parser, vas a poder comprobarlo: `(equal? (parse (a-texto p)) p)` debe dar `#t`.

Pruébalo con `p` y con `q`, no con `r`: ahí falla, y vale la pena entender por qué. `max` lo agregaste tú al árbol en el ejercicio 1, pero el parser que se entregue reconocerá la gramática de esta hoja (`Num`, `add`, `mul`, `neg`) y nadie le habrá avisado de tu extensión. Extender el AST y extender la herramienta que lo construye son dos trabajos distintos: hacer uno no hace el otro.

Al revés no siempre funciona igual de limpio: si partes de un texto con espacios raros (`"add( 3 , 4 )"`) y lo pasas por el parser y luego por `a-texto`, el texto que recuperas no es idéntico al que empezaste, aunque el árbol sí lo sea. Los espacios se perdieron al construir el árbol, y no se perdió nada importante. Eso es exactamente la diferencia entre sintaxis concreta y sintaxis abstracta.